Mikhail Razakov

Compiler & Program Analysis Engineer

misha13022008@gmail.com
t.me/Micodiy
github.com/Misha1302

Compiler engineer spanning LLVM middle-end transformations, program/static analysis, and compiler infrastructure. At MCST, I implemented an LLVM 22 LICM pass and a conservative interprocedural global-state promotion pass; at ISP RAS, I contribute to SharpChecker, an industrial C#/.NET static-analysis platform. Independent work covers Roslyn fixed-point data flow, SSA/x86-64 code generation and register allocation, and deterministic language composition.

Professional compiler / analysis

MCST: LLVM passes, loop/interprocedural analysis, and legality; ISP RAS: SharpChecker C#/.NET static analysis.

Compiler architecture

UniversalToolchain: typed artifacts; dependencies/conflicts, provider selection, pass order, artifact routes, and backends compile into an immutable LanguagePlan before execution.

Verification & backends

Conservative rejection paths, LLVM Verifier and before/after execution; interpreter/CIL parity; a separate x86-64 codegen/register-allocation pipeline.

EXPERIENCE

2026 — present

ISP RAS — Static Analysis Engineer

- Contribute to SharpChecker, ISP RAS's industrial static-analysis platform for C#/.NET.

July — August 2026

MCST — Compiler Engineering Intern

- Implemented an LLVM LICM pass with loop analysis, side-effect/speculative-safety checks, and hoisting of loop-invariant instructions to preheaders.
- Built a conservative interprocedural LLVM pass for localizing induction-like integer globals: APInt affine evolution, transitive call effects, dominance/must-execute legality, and explicit synchronization around known calls; unsupported cases fail closed.

SELECTED RESEARCH & ENGINEERING

LLVM Interprocedural Global IV Optimization

A conservative LLVM 22 prototype/MVP with an actual IR transform; 29 positive/negative regression cases cover verifier validity, idempotence, and before/after execution.

UniversalToolchain

Current exact test manifest: 1,324/1,324. In a frozen-corpus verifier experiment, B2 detected 32/32 primary and 10/10 challenge operators with 0/100 false positives on valid controls; four post-freeze holdouts were checked separately.

DerefAfterNullAnalyzer

Diagnostics for potentially unsafe dereference paths over the real CFG, with successor reprocessing until the data-flow state reaches a fixed point.

x86-64 Codegen & Register Allocation Lab

A reference interpreter and differential execution check generated-code semantics; the allocator assignment contract is verified independently.

SKILLS

Compilers / LLVM

C++23, LLVM 22, LLVM IR, optimization passes, interprocedural analysis, CFG/SSA, dominators, loop analysis, APInt, transformation legality, LLVM Verifier, CMake, Linux

Program / Static Analysis

C#, .NET, Roslyn ControlFlowGraph, worklist/fixed-point data flow, edge-sensitive states, branch refinement, joins, loops/back edges, conservative analysis

Backend / Code Generation

Rust, SSA-like IR, liveness, interference, deterministic register allocation, phi lowering, SysV x86-64, differential execution

Compiler Infrastructure & Verification

typed artifact/component contracts, deterministic planning/pass ordering, provider/capability resolution, exact runtime binding, idempotence and differential checks

EDUCATION

HSE University — Software Engineering, 2026–2030

RECOGNITION

Accepted technical talk on UniversalToolchain's architecture; LangDev'26 takes place October 8–9, 2026 in Málaga.

Moscow / remote